

CHAPTER 13

13. NestJS 인증 — 회원가입 · 로그인 · JWT



Node.js · Express · NestJS 강의 · 2026

9장에서 Express로 인증 게시판을 만들었다. 비밀번호를 `bcrypt` 로 해시하고, 로그인하면 `jwt.sign` 으로 토큰을 발급하고, 보호 라우트 앞에 `auth` 미들웨어를 끼워 `Authorization: Bearer` 헤더를 직접 검사했다. 동작은 같다. 이 장에서는 똑같은 흐름을 **NestJS답게** 다시 짠다.

NestJS답다는 건 직접 짜던 부분을 프레임워크의 표준 자리에 넣는다는 뜻이다. 토큰 발급은 `@nestjs/jwt` 의 `JwtService` 가, 토큰 검증은 **Passport JWT 전략**이, 라우트 보호는 함수 미들웨어가 아니라 **가드(Guard)** 가 맡는다. 9장에서 한 함수 안에 뒤섞여 있던 일이 역할별로 흩어지고, 각 자리는 NestJS가 정한 규격을 따른다. 사용자 저장은 12장에서 붙인 Prisma를 그대로 쓴다.

이 자료는 **개념 예습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- 비밀번호를 `bcrypt` 로 해시해 저장하고(평문 금지), 로그인 때 `bcrypt.compare` 로 대조하는 법
- `@nestjs/jwt` 의 `JwtService` 로 토큰을 발급하고, `JwtModule.register` 로 시크릿·만료시간을 설정하는 법
- **Passport JWT 전략**이 Bearer 토큰을 검증하고 `req.user` 를 채우는 구조
- **가드**(`@UseGuards(JwtAuthGuard)`)로 라우트를 막는 법 — 9장의 `auth` 미들웨어가 NestJS에서 어디로 갔는지
- 시크릿을 한 곳에서 관리해야 하는 이유

흐름은 **회원가입** → **로그인** → **토큰 발급** → **가드로 보호된 라우트** 순서로 한 줄로 이어진다. 내용은 세 주제로 나뉘고, 아래 주제 맵을 그 순서로 읽으면 된다.

주제	핵심
발급	비밀번호 해시 · 검증 · JWT 서명
검증	Passport JWT 전략으로 토큰 확인 → <code>req.user</code>
보호	가드로 라우트 차단 · 통과한 요청만 핸들러로

핵심 개념 — 인증의 일을 세 자리로 나눈다

9장에서는 인증의 모든 일이 한 파일 안에 있었다. 가입 핸들러, 로그인 핸들러, `auth` 미들웨어, 시크릿 상수가 전부 한 파일에 모여 있었다. 작은 예제에서는 그게 읽기 편하다. 하지만 규모가 커지면 "토큰을 발급하는 곳"과 "토큰을 검증하는 곳"과 "라우트를 막는 곳"이 뒤섞여 손대기 어려워진다.

NestJS는 이 셋을 명확히 분리한다.

- **발급** — 인증 서비스가 `JwtService.sign` 으로 토큰을 만든다. "로그인이 성공했으니 토큰을 준다"는 판단은 비즈니스 로직이므로 서비스의 일이다.
- **검증** — JWT 전략이 들어온 토큰의 서명·만료를 확인하고, 통과하면 `validate` 가 돌려준 값이 `req.user` 가 된다. 이걸 Passport라는 표준 인증 라이브러리의 규격을 따른다.
- **보호** — 인증 가드가 라우트 앞을 막는다. 가드가 "통과/차단"만 정하고, 통과한 요청만 핸들러로 보낸다.

Passport 전략 — "어떻게 검증할지"의 규격

Passport는 Node 진영의 표준 인증 라이브러리다. "사용자를 어떻게 확인할지"를 **전략(strategy)** 이라는 단위로 끼워 넣는다. 아이디·비밀번호로 확인하면 `local` 전략, 구글 로그인이면 `google` 전략, 그리고 우리가 쓸 JWT 토큰 검증은 `jwt` 전략이다.

전략 하나는 두 가지를 정한다. 어디서 자격 증명을 꺼낼지(JWT 전략이면 `Authorization` 헤더의 Bearer 토큰)와, 검증이 끝난 뒤 무엇을 `req.user` 에 넣을지(`validate` 의 반환값)다. 9장에서 `auth` 함수가 헤더를 잘라 `jwt.verify` 를 부르고 `req.user` 를 채우던 그 일을, JWT 전략이 규격에 맞춰 대신한다.

가드 — 라우트 앞을 지키는 문지기

가드(Guard) 는 핸들러가 실행되기 전에 "이 요청을 통과시킬지" 한 가지만 판단한다. 통과면 핸들러로 보내고, 아니면 `401` 로 막는다. 9장의 `auth` 미들웨어와 목적이 같다. 다른 점은 미들웨어처럼 (`req`, `res`, `next`) 를 직접 받는 함수가 아니라, NestJS가 정한 가드 인터페이스를 따르는 클래스이고, `@UseGuards(. )` 데코레이터로 라우트에 붙인다는 것이다.

이 장의 가드는 그 판단을 직접 하지 않고 Passport에 위임한다. `AuthGuard("jwt")` 를 상속하면, 가드가 호출될 때 `jwt` 전략을 실행한다. 전략이 검증에 성공하면 통과, 실패하면 자동으로 `401` 이다.

콜아웃 — 미들웨어와 가드는 무엇이 다른가 9장의 `auth` 는 Express 미들웨어였다. 모든 요청에 걸 수도, 특정 라우트에만 걸 수도 있었지만 "라우트를 보호한다"는 의미가 코드에 드러나지 않았다. NestJS의 가드는 역할이 이름과 데코레이터에 박혀 있다. `@UseGuards(JwtAuthGuard)` 를 보면 "이 라우트는 인증이 필요하다"가 한눈에 읽힌다.

준비

이 장은 12장과 마찬가지로 로컬 PostgreSQL과 Prisma를 쓴다. 먼저 이 챕터용 빈 데이터베이스를 만든다.

```
createdb nest_auth
```

`createdb` 명령이 없다면 PostgreSQL이 설치돼 있지 않거나 경로에 없는 것이다. 12장의 준비 절을 참고해 설치한다.

다음으로 의존성을 설치하고 환경변수 파일을 만든다. 예제 폴더에서 `npm install` 로 의존성을 설치하고, `.env.example` 을 `.env` 로 복사한다.

`.env` 에는 채워야 할 키가 둘 있다. `.env.example` 이 그 틀이다.

```
# USER:PASSWORD 를 본인 로컬 PostgreSQL 계정으로 바꾸세요.
DATABASE_URL="postgresql://USER:PASSWORD@localhost:5432/nest_auth?schema=public"

# JWT 서명 시크릿 (운영에서는 길고 무작위한 값으로)
JWT_SECRET="dev-secret-change-me"

# 앱 포트 (선택, 기본 3000)
PORT=3000
```

- `DATABASE_URL` — `USER:PASSWORD` 를 본인 PostgreSQL 계정으로 바꾸고, 데이터베이스 이름은 방금 만든 `nest_auth` 로 둔다.
- `JWT_SECRET` — 토큰에 서명하고 검증할 때 쓰는 비밀 키다. 이 값이 노출되면 누구나 가짜 토큰을 만들 수 있으므로, 운영에서는 길고 무작위한 값으로 바꾼다. 9장에서 코드에 박아 두었던 `SECRET = "my-secret-key"` 가 여기로 빠져나왔다.

스키마를 데이터베이스에 반영한다.

```
npx prisma migrate dev--name init
```

이 명령은 스키마를 보고 `users` 테이블을 만들고, Prisma 클라이언트 타입까지 생성한다. 이제 서버를 띄운다.

```
npm run start:dev      # → http://localhost:3000 (Swagger: /docs)
```

포트는 기본 `3000` 이다. `/docs` 로 들어가면 Swagger 문서가 뜨고, 거기서 바로 요청을 보내볼 수도 있다.

사용자 모델 — 인증에 필요한 최소 스키마

```
model User {
  id          Int          @id @default(autoincrement())
  email       String       @unique
  password    String       // bcrypt 해시 (평문 아님)
  name        String
  createdAt   DateTime     @default(now())
}
```

인증만 다루는 장이라 모델은 `User` 하나로 충분하다. 두 가지만 짚자.

- `email` 에 `@unique` 가 붙어 있다. 같은 이메일로 두 번 가입할 수 없다는 규칙을 데이터베이스 차원에서 막는다. 회원 가입 코드에서도 한 번 더 검사하지만, 동시에 두 요청이 들어오는 경우까지 막는 건 이 제약이다.

- `password` 필드에는 비밀번호 자체가 아니라 **bcrypt** 해시 문자열이 들어간다. 평문 비밀번호는 데이터베이스 어디에도 저장하지 않는다. 주석이 그 점을 못 박는다.

사용자 저장 — 넣고 빼는 일만 담당한다

사용자를 데이터베이스에 넣고 빼는 일은 사용자 서비스가 맡는다. Prisma를 주입받아 세 가지만 한다.

```
// 사용자 저장만 담당 (비밀번호 해싱·검증은 인증 서비스의 몫).
@Injectable()
export class UsersService {
  constructor(private readonly prisma: PrismaService) {}

  create(data: { email: string; name: string; password: string }) {
    return this.prisma.user.create({ data });
  }

  findByEmail(email: string) {
    return this.prisma.user.findUnique({ where: { email } });
  }

  findById(id: number) {
    return this.prisma.user.findUnique({ where: { id } });
  }
}
```

이 서비스는 데이터베이스에 사용자를 넣고 빼는 일만 한다. 비밀번호를 해시하거나 토큰을 발급하는 인증 로직은 전혀 없다. 그 일은 인증 서비스가 한다.

이렇게 나눠 두면 역할이 분명해진다. 사용자 서비스는 "사용자가 있다/없다, 만든다"만 알고, 그 사용자를 어떻게 인증할지는 모른다. `create` 에 넘기는 `password` 도 이미 해시된 값이라는 걸 이 서비스는 신경 쓰지 않는다. 해시는 부르는 쪽의 책임이다.

`findByEmail` 은 로그인할 때 이메일로 사용자를 찾는 데 쓰고, 회원가입 때 중복을 검사하는 데도 쓴다. `findUnique` 는 `@unique` 컬럼으로 한 건을 찾을 때 쓰는 Prisma 메서드다. 없으면 `null` 을 돌려준다.

입력 검증 규격 — DTO

회원가입과 로그인은 받는 값이 다르므로 검증 규격(DTO)도 둘이다. 먼저 회원가입.

```
export class RegisterDto {
  @ApiProperty({ example: "alice@example.com" })
  @IsEmail()
  email: string;

  @ApiProperty({ example: "secret123" })
  @IsString()
  @MinLength(6)
  password: string;

  @ApiProperty({ example: "앨리스" })
  @IsString()
  @MinLength(2)
  name: string;
}
```

`@IsEmail()`, `@MinLength(6)` 같은 데코레이터가 검증 규칙이다. 이메일 형식이 아니거나 비밀번호가 6자 미만이면, 핸들러에 당기도 전에 `400` 으로 걸러진다. 이 검증을 실제로 돌리는 건 앱 부트스트랩에서 전역으로 등록한 `ValidationPipe` 다.

```
app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
```

`whitelist: true` 는 DTO에 선언하지 않은 필드를 요청 본문에서 잘라낸다. 클라이언트가 `password` 와 함께 엉뚱한 필드를 보내도, 선언된 세 개만 남는다. 9장에서 핸들러 안에서 `if (!name || !email || !password)` 로 일일이 검사 하던 일을, NestJS에서는 DTO 데코레이터와 파이프가 대신한다.

로그인 규격은 더 단순하다.

```
export class LoginDto {
  @ApiProperty({ example: "alice@example.com" })
  @IsEmail()
  email: string;

  @ApiProperty({ example: "secret123" })
  @IsString()
  password: string;
}
```

로그인에는 비밀번호 길이 제한(`@MinLength`)이 없다. 가입 규칙이 바뀌어도 이미 가입한 사람은 로그인해야 하므로, 검증 을 느슨하게 둔다. 비밀번호가 맞는지는 길이가 아니라 `bcrypt.compare` 가 판단한다.

시크릿 단일 출처

```
// JWT 시크릿을 "한 곳"에서 관리한다 (서명과 검증이 같은 값을 써야 하므로).
// 한쪽만 바뀌면 모든 토큰이 401이 되는 사고를 막는다. 운영에선 .env 로 분리.
export const jwtConstants = {
  secret: process.env.JWT_SECRET?? "dev-secret-change-me",
};
```

작아 보이지만 이 장에서 가장 중요한 자리다. JWT는 **같은 시크릿으로 서명하고 같은 시크릿으로 검증**해야 한다. 토큰을 만드는 곳(`JwtModule`)과 토큰을 검증하는 곳(JWT 전략), 두 자리가 시크릿을 따로 들고 있으면 한쪽만 바꿨을 때 멀쩡히 발급한 토큰이 전부 401 이 된다.

그래서 시크릿을 `jwtConstants.secret` 한 곳에 두고, 두 자리가 모두 이 값을 가져다 쓴다. 단일 출처를 만들어 어긋날 여지를 없앤다.

`process.env.JWT_SECRET?? "dev-secret-change-me"` 는 `.env` 에 값이 있으면 그걸 쓰고, 없으면 개발용 기본값을 쓴다는 뜻이다. 단, `process.env` 가 채워지려면 `.env` 가 먼저 로드돼 있어야 한다. 그 일은 앱 진입점 맨 윗줄이 한다.

```
import "dotenv/config"; // .env → process.env (시크릿을 읽기 전에 먼저 로드돼야 함)
```

이 `import` 가 맨 위에 있어야 하는 이유가 주석에 있다. 시크릿 상수가 `process.env.JWT_SECRET` 을 읽는 시점에 이미 `.env` 가 로드돼 있어야 하기 때문이다. 순서가 어긋나면 시크릿이 늘 기본값으로 잡힌다.

발급 — 해시 · 검증 · 토큰 서명

인증의 핵심 로직이 모두 인증 서비스에 있다. 회원가입부터 본다.

```
// 회원가입 — 비밀번호는 bcrypt로 해시해서 저장한다(평문 금지).
async register(dto: RegisterDto) {
  const exists = await this.userService.findByEmail(dto.email);
  if (exists) throw new ConflictException("이미 가입된 이메일입니다.");

  const hashed = await bcrypt.hash(dto.password, 10);
  const user = await this.userService.create({
    email: dto.email,
    name: dto.name,
    password: hashed,
  });

  const { password, ...result } = user; // 응답에서 해시 제외
  return result;
}
```

순서대로 읽으면 회원가입의 전부다.

- `findByEmail` 로 같은 이메일이 이미 있는지 본다. 있으면 `ConflictException` 을 던진다. NestJS가 이 예외를 잡아 `409 Conflict` 응답으로 바꾼다. 9장에서 `res.status(409).json(. )` 를 직접 부르던 것을, 여기서는 예외를 던지면 프레임워크가 상태코드로 변환한다.
- `bcrypt.hash(dto.password, 10)` — 비밀번호를 해시한다. 두 번째 인자 `10` 은 솔트 라운드다. 숫자가 클수록 해시 계산이 느려져 무차별 대입 공격이 어려워지지만, 그만큼 로그인도 느려진다. `10` 은 그 균형으로 흔히 쓰는 값이다.
- 해시한 값을 `password` 로 넘겨 사용자를 만든다. 데이터베이스에는 평문이 아니라 이 해시가 들어간다.
- `const { password,... result } = user` — 응답에서 `password` (해시)를 떼어낸다. 해시라 해도 클라이언트에 돌려줄 이유가 없다. 구조분해로 `password` 만 빼고 나머지를 `result` 에 모아 그걸 반환한다.

다음은 로그인을 떠받치는 검증 함수다.

```
// 이메일+비밀번호 검증 - bcrypt.compare로 해시와 대조한다.
async validateUser(email: string, password: string) {
  const user = await this.userService.findByEmail(email);
  if (user && (await bcrypt.compare(password, user.password))) {
    const { password: _pw,... result } = user;
    return result;
  }
  return null;
}
```

`bcrypt.compare(평문, 해시)` 가 핵심이다. 해시는 되돌릴 수 없으므로, 저장된 해시를 평문으로 복원해 비교하는 게 아니라 들어온 평문을 같은 방식으로 해시해 대조한다. 그 일을 `compare` 가 한 번에 처리한다. 맞으면 `true` 다.

사용자가 없거나 비밀번호가 틀리면 똑같이 `null` 을 돌려준다. 둘을 구분하지 않는 게 의도다. "이메일은 맞는데 비밀번호가 틀렸다"고 알려주면, 공격자에게 어떤 이메일이 가입돼 있는지 흘리는 셈이다. 9장에서 "사용자가 없거나 비밀번호가 틀리면 똑같이 401"로 처리하던 것과 같은 원칙이다.

```
// 로그인 - 검증 통과 시 JWT 발급.
async login(email: string, password: string) {
  const user = await this.validateUser(email, password);
  if (!user) {
    throw new UnauthorizedException(
      "이메일 또는 비밀번호가 올바르지 않습니다."
    );
  }
  const payload = { sub: user.id, email: user.email };
  return { access_token: this.jwtService.sign(payload) };
}
```

`validateUser` 가 `null` 을 주면 `UnauthorizedException` 을 던진다(401). 통과하면 토큰을 만든다.

- `payload` 는 토큰 안에 담을 내용이다. `sub` 는 JWT 표준에서 "토큰의 주체(subject)", 즉 누구의 토큰인지를 가리키는 자리다. 여기에 사용자 `id` 를 넣는다. `email` 도 함께 담는다.

- `this.jwtService.sign(payload)` — `@nestjs/jwt` 가 시크릿으로 서명한 토큰 문자열을 만든다. 9장의 `jwt.sign({ id, name }, SECRET, { expiresIn: "1h" })` 와 같은 일이다. 다른 점은 시크릿과 만료시간을 여기서 인자로 넘기지 않는다는 것이다. 그 설정은 `JwtModule.register` 에 한 번 적어 두었고, `JwtService` 가 그걸 가져다 쓴다.
- 응답 키는 `access_token` 이다. 9장에서는 `token` 이었다. 이름만 다를 뿐 클라이언트가 받아 두었다가 다음 요청의 `Authorization` 헤더에 실어 보낼 그 토큰이다.

콜아웃 — **payload** 에는 민감 정보를 넣지 않는다 JWT의 페이로드에는 암호화가 아니라 서명일 뿐이다. 누구나 토큰을 디코드하면 `sub` 와 `email` 을 읽을 수 있다. 서명은 "내용이 위조되지 않았다"를 보장할 뿐, "내용을 숨긴다"가 아니다. 그래서 비밀번호 해시 같은 건 절대 페이로드에 담지 않는다. 식별에 필요한 최소한(`id`, `email`)만 넣는다.

검증 — 토큰을 확인하는 JWT 전략

```
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      ignoreExpiration: false,
      secretOrKey: jwtConstants.secret, // 서명 시크릿과 동일해야 함
    });
  }

  // 토큰 서명이 유효하면 호출된다. 반환값이 곧 req.user 가 된다.
  // 토큰 자체가 신뢰 근거이므로 DB 조회 없이 payload에서 바로 구성.
  // payload = { sub, email } (sub = user.id)
  async validate(payload: any) {
    return { id: payload.sub, email: payload.email };
  }
}
```

9장에서 `auth` 미들웨어가 한 일 — 헤더에서 토큰을 꺼내고, 서명과 만료를 확인하고, 통과하면 `req.user` 를 채우던 일 — 이 전부가 JWT 전략으로 옮겨 왔다. `PassportStrategy(Strategy)` 를 상속하고 두 부분을 채운다.

생성자의 `super({, })` 는 전략의 설정이다.

- `jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken()` — 토큰을 어디서 꺼낼지다. `Authorization: Bearer <토큰>` 헤더에서 `Bearer` 뒤를 잘라낸다. 9장에서 `header.slice(7)` 로 직접 자르던 코드가 이 한 줄로 정리된다.
- `ignoreExpiration: false` — 만료를 무시하지 않는다. 즉 만료된 토큰은 검증에서 떨어진다. 토큰은 `1h` 뒤에 만료되도록 발급했으므로, 한 시간이 지나면 자동으로 거부된다.

- `secretOrKey: jwtConstants.secret` — 검증에 쓸 시크릿이다. 발급에 쓴 것과 같은 `jwtConstants.secret` 이다. 시크릿 단일 출처가 여기서 한 번 더 확인된다.

`validate(payload)` 는 서명과 만료가 통과한 다음에 호출된다. 즉 이 함수가 불렀다는 건 **토큰이 이미 유효하다는** 뜻이다. 그래서 여기서는 토큰을 다시 의심하지 않고, 페이로드에서 꺼낸 값으로 `req.user` 를 구성하기만 한다. `{ id: payload.sub, email: payload.email }` 을 돌려주면 그게 `req.user` 가 된다.

여기서 의도적으로 **데이터베이스를 조회하지 않는다**. 토큰의 서명이 유효하다는 것 자체가 "우리가 발급한 진짜 토큰"이라는 신뢰의 근거이기 때문이다. `validate` 안에서 `sub` 로 다시 사용자를 조회하고 없으면 예외를 던지게 짜면, 사용자가 탈퇴했거나 조회가 잠깐 실패했을 때 말썽한 토큰까지 깨질 수 있다. 그런 정책이 꼭 필요할 때만 DB를 태우고, 기본은 페이로드만으로 가볍게 구성한다.

클아웃 — `validate` 의 반환값이 곧 `req.user` 다 컨트롤러의 `req.user` 가 어디서 오는지 헷갈린다면, 답은 항상 `validate` 의 `return` 이다. 여기서 돌려준 객체가 그대로 핸들러의 `req.user` 에 담긴다. `id` 와 `email` 만 넣었으니 `req.user` 에도 그 둘만 들어 있다.

보호 — 라우트를 막는 가드

```
// AuthGuard("jwt") = JWT 전략을 실행하는 가드.
// @UseGuards(JwtAuthGuard)를 붙인 라우트는 유효한 Bearer 토큰이 있어야 통과한다.
@Injectable()
export class JwtAuthGuard extends AuthGuard("jwt") {}
```

내용이 비어 있다. `AuthGuard("jwt")` 를 상속하는 것만으로 할 일이 다 된다. 그 한 줄이 "이 가드가 호출되면 `jwt` 전략을 실행하라"는 뜻이다. 문자열 `"jwt"` 가 앞서 만든 JWT 전략을 가리킨다.

전략이 검증에 성공하면 가드는 요청을 통과시키고 `req.user` 까지 채워 둔다. 실패하면(토큰이 없거나, 서명이 틀렸거나, 만료됐거나) 가드가 자동으로 `401` 을 던진다. 9장에서 `auth` 미들웨어 안에 `try/catch` 로 적던 "유효하지 않으면 `401`" 분기가, 여기서는 가드와 전략이 알아서 처리한다.

그럼 굳이 빈 클래스를 따로 만드는 이유는 무엇인가. `@UseGuards(AuthGuard("jwt"))` 처럼 직접 써도 동작하지만, 이름을 가진 `JwtAuthGuard` 로 감싸 두면 라우트에서 의도가 또렷이 읽히고, 나중에 가드에 동작을 더해야 할 때 한 곳만 고치면 된다.

세 엔드포인트 — 컨트롤러

```
@ApiTags("auth")
@Controller("auth")
export class AuthController {
  constructor(private readonly authService: AuthService) {}

  @Post("register") // POST /auth/register
  register(@Body() dto: RegisterDto) {
    return this.authService.register(dto);
  }

  @Post("login") // POST /auth/login → { access_token }
  login(@Body() dto: LoginDto) {
    return this.authService.login(dto.email, dto.password);
  }

  // 보호된 라우트 — 유효한 Bearer 토큰이 있어야 통과한다.
  @UseGuards(JwtAuthGuard)
  @ApiBearerAuth()
  @Get("profile") // GET /auth/profile
  profile(@Request() req) {
    // JWT 전략의 validate가 반환한 값이 req.user 에 들어있다.
    return req.user;
  }
}
```

`@Controller("auth")` 덕분에 세 라우트의 경로 앞에 모두 `auth` 가 붙는다. 컨트롤러는 얇다. 입력을 받아 서비스에 넘기고 결과를 돌려줄 뿐, 인증 로직은 한 줄도 없다.

- `register` — `@Body()` `dto: RegisterDto` 로 본문을 받는다. `ValidationPipe` 가 DTO 규칙으로 검증한 뒤에야 이 핸들러에 닿으므로, 핸들러 안에서는 형식을 다시 검사하지 않는다. 인증 서비스의 `register(dto)` 만 부른다.
- `login` — 마찬가지로 `LoginDto` 를 받아 인증 서비스의 `login` 에 이메일과 비밀번호를 넘긴다. 결과 `{ access_token }` 이 그대로 응답이 된다.
- `profile` — 여기에 세 데코레이터가 함께 붙는다. `@UseGuards(JwtAuthGuard)` 가 가드로 라우트를 막고, `@Get("profile")` 이 경로를 정하고, `@ApiBearerAuth()` 는 Swagger 문서에 "이 라우트는 Bearer 토큰이 필요하다"를 표시한다. 핸들러 본문은 `return req.user` 한 줄이다. 토큰이 유효해 가드를 통과하면, 전략의 `validate` 가 채운 `req.user` 가 응답으로 나간다.

이 장의 결론은 `profile` 핸들러에 있다. 핸들러에는 인증 코드가 전혀 없다. 토큰 검사도, 헤더 파싱도, `401` 분기도 모두 가드와 전략으로 빠졌고, 핸들러는 통과한 요청만 받아 자기 일을 한다.

조각을 묶는 모듈

```
@Module({
  imports: [
    UsersModule,
    PassportModule,
    JwtModule.register({
      secret: jwtConstants.secret, // 전략과 동일 출처
      signOptions: { expiresIn: "1h" },
    }),
  ],
  controllers: [AuthController],
  providers: [AuthService, JwtStrategy],
})
export class AuthModule {}
```

지금까지 만든 조각들이 인증 모듈에서 하나로 묶인다.

- `imports` 에 `UsersModule` 을 넣어야 인증 서비스가 사용자 서비스를 주입받을 수 있다. `UsersModule` 은 `exports` 로 사용자 서비스를 내보내 두었다.
- `PassportModule` 은 Passport 기반 가드-전략을 쓰기 위한 준비다.
- `JwtModule.register({})` — 여기서 토큰 설정을 **한 번** 정한다. `secret` 은 `jwtConstants.secret` (전략과 같은 출처), `signOptions: { expiresIn: "1h" }` 로 만료를 1시간으로 둔다. 인증 서비스의 `jwtService.sign(payload)` 이 이 설정을 가져다 쓰기 때문에, 발급 코드에서는 시크릿과 만료를 다시 적지 않는다.
- `providers` 에 `JwtStrategy` 를 등록해야 전략이 살아 있다. 이걸 빠뜨리면 `AuthGuard("jwt")` 가 실행할 전략을 못 찾아 보호 라우트가 깨진다.

`JwtModule` 과 JWT 전략 양쪽이 같은 `jwtConstants.secret` 을 `secret` 으로 가리킨다. 이게 시크릿 단일 출처의 실제 모습이다.

인증 흐름 한눈에

지금까지의 조각이 요청 하나에서 어떻게 이어지는지 정리하면 이렇다.

[회원가입]

POST /auth/register → ValidationPipe(DTO 검증) → 인증 서비스 register
 ↳ findByEmail(중복 검사) → bcrypt.hash(라운드 10) → 사용자 생성 → 해시 뺀 결과 반환

[로그인]

POST /auth/login → 인증 서비스 login
 ↳ validateUser(bcrypt.compare) → 통과 → jwtService.sign(payload) → { access_token }

[보호 라우트]

GET /auth/profile (Authorization: Bearer <token>)
 ↳ JwtAuthGuard → JWT 전략: 헤더에서 토큰 추출 → 서명·만료 검증
 ↳ 통과 → validate(payload) → req.user = { id, email }
 ↳ 실패 → 401
 ↳ 통과한 요청만 profile 핸들러 → return req.user

9장 Express 버전과 무엇이 달라졌나

	9장 (Express)	13장 (NestJS)
토큰 발급	jwt.sign(payload, SECRET, { . }) 직접 호출	JwtService.sign(payload) (설정은 JwtModule)
토큰 검증	auth 미들웨어 안 jwt.verify + try/catch	JWT 전략 (Passport 규격)
라우트 보호	app.post("/posts", auth, handler)	@UseGuards(JwtAuthGuard)
시크릿	코드에 박은 const SECRET	단일 출처 상수 + .env
입력 검증	핸들러 안 if (!email...)	DTO 데코레이터 + ValidationPipe
사용자 저장	인메모리 배열	Prisma (사용자 서비스)

동작은 같다. 달라진 건 같은 일을 **역할별 자리에 나눠 담았다**는 점이다. 작은 예제에서는 9장이 더 짧고 읽기 쉽지만, 라우트가 늘고 정책이 복잡해질수록 NestJS의 분리가 손대기 쉬운 코드를 만든다.

흔한 실수

- **JwtModule** 과 **JWT 전략**의 시크릿이 다르다. → 발급은 되는데 검증에서 전부 401 . 단일 출처 시크릿을 양쪽이 같이 쓰는지 확인한다. 이 장이 시크릿을 한 곳에 모은 이유가 정확히 이것이다.

- 인증 모듈의 `providers` 에서 `JwtStrategy` 를 빼뜨린다. → 전략이 등록되지 않아 `AuthGuard("jwt")` 가 실행할 전략을 못 찾는다. 보호 라우트가 통째로 깨진다.
- `validate` 에서 DB를 조회하고 없으면 예외를 던진다. → 사용자 탈퇴나 일시적 조회 실패에 멀쩡한 토큰까지 401 이 된다. 꼭 필요한 정책이 아니면 페이로드만으로 구성한다.
- 앱 진입점의 `import "dotenv/config"` 가 맨 위에 없다. → 시크릿을 읽는 시점에 `.env` 가 아직 안 올라와, 시크릿 이 늘 기본값으로 잡힌다.
- 응답에서 `password` (해시)를 안 뺀다. → 해시라도 클라이언트에 돌려줄 이유가 없다. 회원가입·로그인 검증에서 구조 분해로 떼어낸다.
- 요청 때 `Authorization: Bearer` 접두사를 빼뜨린다. → 전략이 토큰을 못 꺼내 401 . 토큰만 달랑 보내면 안 되고 `Bearer <토큰>` 형태여야 한다.

직접 해보기

1. 만료를 짧게 바꿔 본다. `JwtModule.register` 의 `expiresIn` 을 "30s" 로 바꾸고 서버를 재시작한 뒤, 로그인해 서 받은 토큰으로 `GET /auth/profile` 을 30초 안에 한 번, 30초 뒤에 한 번 호출해 본다. 만료 후에는 401 이 떨어 지는지 확인한다. (`ignoreExpiration: false` 가 일하는 모습이다.)
2. `/users/me` 같은 보호 라우트에서 실제 사용자 정보를 돌려준다. `profile` 은 `req.user (= { id, email } )` 만 돌려준다. 이걸 `UserService.findById(req.user.id)` 로 데이터베이스에서 최신 정보를 조회해 `name` 과 `createdAt` 까지 담아 돌려주도록 핸들러를 고쳐 본다. (단, `password` 는 빼고.)
3. 잘못된 토큰으로 막히는지 확인한다. 로그인으로 받은 토큰의 가운데 글자 하나를 임의로 바꿔 `Authorization` 헤더 에 넣고 `GET /auth/profile` 을 호출해 본다. 서명이 깨져 401 이 나는지 본다. 시크릿을 모르면 토큰을 위조할 수 없다는 걸 직접 확인하는 실습이다.

정리

- 인증의 일을 세 자리로 나눴다. 발급은 인증 서비스(+ `JwtService`), 검증은 JWT 전략, 보호는 인증 가드.
- 비밀번호는 `bcrypt.hash` (라운드 10)로 해시해 저장하고, 로그인 때 `bcrypt.compare` 로 대조한다. 평문은 데이터 베이스에 남기지 않는다.
- 시크릿은 한 곳에서 관리한다. `JwtModule` 과 JWT 전략이 같은 값을 써야 토큰이 깨지지 않는다.
- 전략의 `validate` 반환값이 곧 `req.user` 다. 토큰 서명이 신뢰 근거이므로 기본은 DB를 타지 않고 페이로드만으 로 구성한다.
- `@UseGuards(JwtAuthGuard)` 한 줄이 9장의 `auth` 미들웨어를 대신한다. 핸들러에는 인증 코드가 남지 않는다.

다음 장(14 · NestJS 캐싱 + 스케줄링)에서는 게시판 도메인으로 두 가지 실무 패턴을 익힌다. 비싼 조회 결과를 Redis에 저장해 DB 부하를 줄이는 캐싱(cache-aside) 과, 주기적으로 통계를 집계하는 `@Cron` 스케줄링이다.